Supplementary Material for AgentISA: Formal Instruction Set Design for LLM Agents

Anonymous Author(s)

Abstract

This document is the supplementary material for *AgentISA: Formal Instruction Set Design for LLM Agents*. It contains the appendices referenced by the main paper: Appendix A (the Turing-completeness construction for Theorem 3.1), Appendix B (the type-safety proof for Theorem 3.2), and Appendix C (the semantic-preservation lemmas and proofs of Theorems 3.4–3.6, with the scope remarks of C.7). Section and theorem numbers follow the main paper.

A Proof of Theorem 3.1 (Turing Completeness)

A.1 Strategy

We prove that AgentISA can simulate any program in the WHILE language over the natural numbers. WHILE programs compute exactly the partial recursive functions—WHILE is Turing equivalent [43, 44]—so a sound simulation of WHILE by AgentISA establishes Turing completeness. The argument is a sufficiency (simulation) argument over instruction classes: the instruction classes of Definition 3.1 must jointly provide unbounded storage, sequential composition, conditional branching, and unbounded iteration. We show the control-flow class $\mathcal{C}$ and the state-management class $\mathcal{S}$ suffice; neither effectful class (the LLM-interaction class $\mathcal{L}$ nor the resource class $\mathcal{R}$) is needed, making the result conservative with respect to the full instruction set.

A.2 Source Language

Let WHILE be the language

$$P ::= x := x{+}1 \mid x := x{-}1 \mid x := y \mid \texttt{skip} \mid \texttt{if } x = 0 \texttt{ then } P_1 \texttt{ else } P_2 \mid \texttt{while } x \neq 0 \texttt{ do } P \mid P_1; P_2$$

over variables ranging over $\mathbb{N}$ (with predecessor cut off at 0), with the standard small-step semantics. Configurations are pairs $\langle P, \sigma \rangle$ with store $\sigma : Var \to \mathbb{N}$.

A.3 Encoding

(i) Store. A WHILE store σ is represented by the AgentISA state $s_\sigma \in \Sigma$, in the component managed by the state-management instructions. Each variable x occupies a dedicated cell; cells hold unbounded naturals, so Σ provides an unbounded store—the crucial ingredient of the sufficiency argument.

(ii) Instruction semantics (as used here). The state-management component of Σ carries, alongside the variable cells, an **evaluation stack**; the five encoding instructions below operate on that stack. Five instructions suffice for the encoding, with the following operational meaning:

- `load` x: pushes the contents of the cell for x onto the evaluation stack;
- `store` x: pops the top of the evaluation stack into the cell for x;
- `update` (f): pops the top of the stack, applies the unary arithmetic function f, and pushes the result; `update`(-1) denotes the *truncated predecessor* $\lambda n.\ \max(n-1, 0)$, matching WHILE's cutoff semantics on $\mathbb{N}$;
- `branch` (b, Q_1, Q_2): pops the top of the stack; if predicate b holds, execute Q_1, else execute Q_2;
- `loop` (Q): execute Q; if its final `branch` returns to the loop head, repeat, else exit—that is, `loop` repeats its body for as long as the body's trailing test succeeds.

(iii) Statements. Each WHILE statement compiles to the AgentISA sequence below (notation defined after the table):

Table 1: Compilation of WHILE statements to AgentISA sequences ($\hat{P}$ is the compilation of P).

WHILE construct	AgentISA compilation
$x := y$	`load` y; `store` x
$x := x + 1$	`load` x; `update`$(+1)$; `store` x
$x := x - 1$	`load` x; `update`(-1); `store` x
`if` $x = 0$ `then` P_1 `else` P_2	`load` x; `branch`$(= 0?, \hat{P}_1, \hat{P}_2)$
`while` $x \neq 0$ `do` P	`loop`(`load` x; `branch`$(\neq 0?, \hat{P}, \epsilon)$)
$P_1; P_2$	$\hat{P}_1 \circ \hat{P}_2$ (sequential composition)

where $\hat{P}$ denotes the compilation of P. The no-op `skip` compiles to the identity instruction ϵ; the `while` row's trailing `branch` exits to ϵ when the guard fails, which is exactly the unfolding identity below.

A.4 Simulation Lemma

Lemma A.1. For every WHILE configuration $\langle P, \sigma \rangle$ and its encoded AgentISA instruction sequence $\hat{P}$ started in state s_σ:

(a) if $\langle P, \sigma \rangle \to \langle P', \sigma' \rangle$, then the AgentISA machine, started at configuration $(\hat{P}, s_\sigma)$ (remaining instruction sequence, Σ-state), reaches the configuration $(\hat{P}', s_{\sigma'})$—remaining sequence $\hat{P}'$ the encoding of the residual program P', state $s_{\sigma'}$ the encoding of σ'—in finitely many steps; and

(b) if $P =$ `skip` (terminal), then $\hat{P}$ terminates.

Proof. By structural induction on P:

- **Assignments** (base cases). Each compiled sequence performs exactly the corresponding store update; (a) holds by executing the two or three instructions in order, (b) trivially.
- **Conditional.** The `branch` instruction evaluates the zero test on the loaded value and transfers control to one of the compiled branch bodies. By the induction hypothesis applied to P_1 and P_2, each branch simulates its WHILE counterpart.

- **Loop.** The WHILE unfolding identity `while x ≠ 0 do P ≡ if x = 0 then skip else (P ; while x ≠ 0 do P)` is simulated by the `loop` instruction's iteration: each iteration loads x and executes the trailing `branch`, which dispatches to $\hat{P}$ when $x \neq 0$ and to ϵ (loop exit) when $x = 0$. By an outer induction on the number of executed iterations (in addition to the structural induction on P announced above—the induction hypothesis applied to P covers each body execution, and the trailing branch discharges the loop condition)—the k-th WHILE iteration is matched by the k-th `loop` iteration.
- **Sequence.** $\hat{P}_1 \circ \hat{P}_2$ executes $\hat{P}_1$ to completion, then $\hat{P}_2$; both simulations compose by the induction hypothesis.

The case arguments above are the substantive reduction steps; expanding each into a fully formal induction over the small-step rules (matching each WHILE step against the transition functions $\delta \in \Delta$ of the corresponding AgentISA instructions) is mechanical and follows the template of textbook simulation arguments [33, 43]. We mark those expansions as routine and leave them to the reader. ■

A.5 Conclusion

By Lemma A.1, every WHILE-computable function is AgentISA-computable. Since WHILE is Turing equivalent—a classical result of computability theory, with WHILE serving as the standard imperative model in semantics texts such as [43]—AgentISA is Turing complete. ■

A.6 Scope Remark

Turing completeness is a deliberately weak property—any language with unbounded storage, branching, and unbounded iteration attains it. Its role here is negative certification: the instruction set does not artificially constrain what agent programs can express, so no expressiveness objection can be raised against the architecture. The claim says nothing about efficiency, and it is established without the LLM-interaction class $\mathcal{L}$, so it is conservative with respect to the full instruction set of Definition 3.1.

B Proof of Theorem 3.2 (Type Safety)

We prove type safety for the pure core by progress and preservation, in the style of Pierce [36]. The proof runs over a **merged judgment system**: the term typing judgment $\vdash e : T$ for λ-terms, and the instruction judgment $\vdash i\ ok[pre(i), post(i)]$ for instructions and their compositions (both defined in B.1). Progress and preservation are each proved by induction over the derivations of this merged system: the λ-cases are the standard TAPL argument on term derivations; the instruction cases are discharged by the standing totality assumption and the per-operator premises of B.1(ii), with a joint runtime invariant threaded through the composition cases.

B.1 Core Calculus

Restrict attention to the pure core of the instruction language: expressions over base types, dependent Π/Σ pairs, recursion (μ), λ-abstraction and application (whose β-reduction rule appears in Section 3.1.2), and the composition operators of Definition 3.6 applied to instructions carrying typed pre/post-conditions. The small-step relation $\rightarrow$ is generated by β-reduction, the context

rule $E[e_1] \to E[e_1']$, and, for each instruction i, its **fixed per-instruction** transition function δ_i. Effectful instructions are treated in B.7.

The two judgment forms:

(i) **Term typing** $\vdash e : T$—the usual rules over base types, extended with the dependent Π/Σ introduction forms of Definition 3.2.

(ii) **Instruction typing** $\vdash i \ ok[pre(i), post(i)]$—read "$i$ is a well-contracted state transformer": $pre(i)$ and $post(i)$ are well-typed predicates over the state type, and the Definition 3.5 obligation $\forall s \vDash pre(i) : \delta_i(s) \vDash post(i)$ is discharged per instruction, offline, by the bounded verification of Section 3.3.3. Instructions are **not values and never stuck**: totality on precondition-satisfying states is a standing assumption—for each instruction i with its transition function δ_i, $\delta_i(s)$ is defined whenever $s \vDash pre(i)$.

Composition rules, for instructions i_1, i_2 carrying contracts:

$$\frac{\vdash i_1 \ ok \qquad \vdash i_2 \ ok \qquad post(i_1) \Rightarrow pre(i_2)}{\vdash i_1 \circ i_2 \ ok[pre(i_1),\, post(i_2)]}$$

$$\frac{\vdash i_1 \ ok \qquad \vdash i_2 \ ok \qquad pre(i_1) \Leftrightarrow pre(i_2) \quad \text{disjoint state footprints}}{\vdash i_1 \parallel i_2 \ ok[pre(i_1),\, post(i_1) \wedge post(i_2)]}$$

$$\frac{\vdash i_1 \ ok \qquad \vdash i_2 \ ok \qquad post(i_1)\big|_{\text{trigger}} \Rightarrow pre(i_2)}{\vdash i_1 \rhd i_2 \ ok[pre(i_1),\, post(i_1) \wedge (\text{trigger} \Rightarrow post(i_2))]}$$

$$\frac{\vdash i \ ok \qquad \text{guard closed under } post(i) \qquad \text{guard} \Rightarrow pre(i)}{\vdash i^* \ ok[\text{guard},\, post(i)]}$$

Two premises deserve note. The $\rhd$ premise is evaluated **on the outcome of i_1 that triggers i_2**: i_1's execution can otherwise destroy $pre(i_2)$, so the implication the runtime invariant needs is $post(i_1)$ restricted to the triggering outcome, not $pre(i_1) \Rightarrow pre(i_2)$ on the initial state. The i^* premise adds guard $\Rightarrow pre(i)$ because the loop body re-enters: on every re-iteration the guard holds, so the body's precondition must hold whenever the loop has not terminated. These rules are generated from—do not add to—the semantics of Definition 3.6.

B.2 Canonical Forms

If $\vdash v : T$ and v is a value, then:

- $T \in \{Bool, Int, String, Float\}$ implies v is a literal of that type;
- $T = \Pi(x : T_1).T_2$ implies $v = \lambda x{:}T_1.\, e$ with $\Gamma, x{:}T_1 \vdash e : T_2$;
- $T = \Sigma(x : T_1).T_2$ implies $v = (v_1, v_2)$ with the components typed at the corresponding dependent types;
- $T = \mu X.T$ implies v is a rolled recursive value;
- no instruction or instruction composition is a value (B.1(ii)): instructions are contracted state transformers, not term-level values.

Proof. By inspection of the typing rules: each type has exactly one value form, and the syntax of values is defined per type, so the case analysis is exhaustive. This is the standard canonical-forms lemma [36]. ■

B.3 Lemma (Substitution)

If $\Gamma, x{:}T' \vdash e : T$ and $\vdash v : T'$, then $\Gamma \vdash e[x := v] : T$.

Proof. By induction on the derivation of $\Gamma, x{:}T' \vdash e : T$. The non-dependent cases are the case analysis of Pierce [36], which we do not duplicate. The dependent Π/Σ cases additionally require synchronizing the substitution into the second component's type ($T_2[x := v]$), as in the β-case of B.5—the standard dependent-type substitution bookkeeping, not covered by the non-dependent treatment in [36]. ■

B.4 Lemma (Progress)

If $\vdash e : T$, then e is a value or there exists e' with $e \to e'$.

Proof. By induction on the derivation. The λ-term cases:

- **Literals and** λ are values.
- **Application** $e_1\, e_2$ with $e_1 : \Pi(x : T_1).T_2$ and $e_2 : T_1$: by the induction hypothesis, either e_1 steps (then the context rule applies) or e_1 is a value, in which case B.2 gives $e_1 = \lambda x.\, e$; similarly e_2 either steps or is a value; when both are values, β-reduction applies. There is no stuck case, because the Π-type of e_1 and B.2 jointly guarantee a λ in value position.

The instruction cases admit no stuck state by construction. Instructions step by the standing totality assumption of B.1(ii) whenever their precondition holds. Compositions maintain the runtime invariant **each reached component's precondition holds**, per operator:

- **Sequencing** $\circ$**:** $pre(i_1)$ holds initially (the composition's contract); after i_1 executes, $post(i_1) \Rightarrow pre(i_2)$ re-establishes the invariant for i_2.
- **Parallel** $\|$**:** disjoint state footprints keep each component's precondition valid throughout the interleaving—neither component writes what the other's precondition or postcondition reads.
- **Guarded** $\triangleright$**:** on the triggering outcome of i_1, the strengthened premise $post(i_1)\big|_{\text{trigger}} \Rightarrow pre(i_2)$ re-establishes $pre(i_2)$; on the non-triggering outcome, i_2 does not run.
- **Iteration** i^***:** the body re-enters only while the guard holds, and the premise guard $\Rightarrow pre(i)$ makes each re-entry legitimate; when the guard fails, the loop is terminal.

The invariant is threaded jointly with B.5: each individual step preserves the invariant by the per-operator argument above. For iteration, either the guard is false (terminal) or the body's precondition holds by guard $\Rightarrow pre(i)$, and the body steps by the induction hypothesis on the component derivation. ■

B.5 Lemma (Preservation)

If $\vdash e : T$ and $e \to e'$, then $\vdash e' : T$.

Proof. By induction on the derivation, with case analysis on the step:

- β**:** $(\lambda x.\, e)\, v \to e[x := v]$. The application rule types the redex at $T_2[x := v]$; by B.3, the substituted term $e[x := v]$ types at $T_2[x := v]$, which is exactly the type the application rule assigned (the dependent-type bookkeeping; when x does not occur in T_2 this degenerates to the usual $T_2 = T$).
- **Context:** $E[e_1] \to E[e_1']$ with $e_1 \to e_1'$. By the induction hypothesis $e_1' : T_1$; an induction on the structure of E re-types $E[e_1']$ at the original type.

- **Instruction transitions:** the instruction's *ok* judgment (B.1(ii)) certifies that δ_i establishes $post(i)$ from any $pre(i)$-satisfying state (discharged offline by the bounded verification of Section 3.3.3), so the reduct re-types at the postcondition.
- **Composition operators:** each operator's step is typed by its B.1(ii) rule; the reduct's contract follows from the operator's stated conclusion (for $\circ$, $post(i_2)$; for $\|$, the conjunction; for $\rhd$, the trigger-restricted conjunction; for i^*, $post(i)$), by the induction hypothesis on the component derivations.

The remaining instruction-class cases repeat this pattern with class-specific invariants; they are routine and omitted (the template is the standard preservation argument for the corresponding constructs [36]). ■

B.6 Theorem 3.2

By B.4, a well-typed closed term is a value or takes a step; by B.5, every step preserves typing. Hence no well-typed term reaches a stuck state: well-typed programs in the pure core cannot go wrong. ■

B.7 Scope Remarks

Two caveats state the theorem's boundaries honestly. First, B.4's appeal to totality of transition functions on precondition-satisfying states is an assumption about instruction *implementations*, not something the type system alone discharges. The per-instruction mechanism intended to discharge it is the bounded verification of Section 3.3.3, which is now shipped code: the checker mechanically discharges the obligation for pure instructions on the state spaces it can enumerate—exhaustively for small enumerable domains, by deterministic bounded sampling beyond—while side-effectful instructions and state spaces beyond the enumeration bound remain covered by hand argument and unit tests rather than by the mechanical check. The theorem is conditional on the obligation being (or becoming) true of each instruction, to the extent stated. Second, effectful instructions (LLM calls, resource allocation) can fail at runtime for reasons outside the type system (network errors, exhausted capacity); Theorem 3.2 covers the pure core and effectful instructions with checkable preconditions. Runtime failure handling for the remaining cases is the responsibility of the fallback machinery of Section 5.6, not of types.

C Proofs of Theorems 3.4–3.6

C.1 Denotational Setup

Fix the state space Σ (Definition 3.1's state space—not the dependent-type constructor Σ of Definition 3.2). For a pure instruction i, let $[\![i]\!] : \Sigma \rightharpoonup \Sigma$ denote its denotation, a partial function total on states satisfying $pre(i)$. Program denotation is defined by structural recursion on the composition algebra of Definition 3.6:

- $[\![i]\!]$ for atomic instructions (given);
- $[\![p_1 \circ p_2]\!] = [\![p_2]\!] \circ [\![p_1]\!]$ (note the order: p_1 executes first);
- $[\![p_1 \parallel p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ for pure p_1, p_2 with disjoint state footprints (as the $\|$-typing rule requires)—the order is immaterial because updates to disjoint cells commute on their domains;

- $[\![p_1 \rhd p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ when p_1's outcome at σ is the trigger, and $[\![p_1]\!](\sigma)$ otherwise (the denotation is defined on $\Sigma +$ outcome, with p_2 applied conditionally on the trigger tag);
- $[\![p^*]\!]$ is the least fixed point of the corresponding functional.

Write $p \equiv q$ iff $[\![p]\!] = [\![q]\!]$ on all states where both are defined.

C.2 Lemma (Atomic Rewrite Soundness)

Each rewrite rule implemented by the optimizer (Section 3.4.4) preserves denotation.

(i) Fusion. $i_1 \circ i_2 \to i_{12}$, with $pre(i_{12}) = pre(i_1)$, $post(i_{12}) = post(i_2)$, and i_{12}'s body stipulated to compute $[\![i_2]\!] \circ [\![i_1]\!]$ (intermediate bindings named, so the composition is explicit). The denotational claim is then an instance of the definition of $\circ$ (C.1): a body computing $[\![i_2]\!] \circ [\![i_1]\!]$ denotes exactly $[\![i_1 \circ i_2]\!]$ (the associativity of function composition is what fuses *three or more* consecutive instructions). **Two honest boundaries.** First, this is an equality *of the rewrite algebra*: it holds of the fused form whose body computes the composite. The shipped *formal compiler* (the static path of the Section 5.3 benchmarks) produces a formal fused *node* (merged metadata and dependencies) without generating an executable composite body—code generation for fused nodes is not implemented—so what those benchmarks measure is the structural fusion of the formal stream, and the executable-fusion claim would additionally require the codegen step. The *Instruction-level* merge path (the dynamic optimizer of Sections 4.2 and 5.4) does carry an executable body—the `CompositeInstruction` replay construction—and its differential property (composite execution equals constituent-sequence execution; the fusion gate refuses producer–consumer pairs) is pinned by unit tests and exercised by the trace-replay identity of Section 5.4 (5/5 seeds). A fused *single* executable body distinct from replay remains specified. Second, the compiler applies fusion only when output types of i_1 match input types of i_2 and both members are pure, since denotational equality of the fused form presupposes order-insensitivity of intermediate effects, and the shipped gate achieves it via purity (the pure or order-insensitive alternative is specified, not implemented).

(ii) Dead-code elimination. Instruction i is removed from p when $Def(i) \cap Live_{out}(i) = \emptyset$, with liveness the standard fixpoint of $Live_{in}(i) = Use(i) \cup (Live_{out}(i) - Def(i))$ [33]. Claim: if i is dead and side-effect-free, then $p \equiv p \setminus \{i\}$. Proof sketch: by induction on the suffix program following i, its denotation depends only on the live variables at i's position; i writes none of them, so removing i cannot change any value subsequently read. The side-effect-freeness gate matches the purity restriction of Definition 3.6 (Property 2 and the Remark following it)—instructions declared `side_effects=True` are never eliminated.

(iii) Peephole combining. Each rule $(s_1, \ldots, s_k) \to s'$ carries the verification obligation $\forall \sigma : [\![s_1 \circ \ldots \circ s_k]\!](\sigma) = [\![s']\!](\sigma)$, to be discharged at rule-definition time (Section 4.1.3). Candidate instances, *illustrative only—the shipped optimizer registers none of them*: `LOAD A; STORE B` composes to `MOVE A→B` (degenerate to the identity when A = B, since the value read is the value written; the general rule of Section 3.4.4); consecutive `QUERY` instructions with identical (instruction, params) pairs collapse to one query whose result is reused (the batch-QUERY rule of Section 3.4.4, specialized to identical pairs) (an obligation about a *compile-time* rewrite; the caching measured in Section 5.2 is a different, *runtime* mechanism—a result cache consulted at execution time—and its numbers do not evidence this rule); consecutive `CONTEXT_UPDATE`s compose to a single update by associativity of the update function.

Proof. Each case reduces to the construction of the rewritten instruction and the definition of liveness. The inner inductions—on the pair of bodies for (i), on suffix programs for (ii)—are

routine and left to the reader. ■

C.3 Lemma (Compositionality)

Let T be a program transform with $[\![T(p)]\!] = [\![p]\!]$ for all p. Then for any context $C[\cdot]$ built from the composition operators, $[\![C[T(p)]]\!] = [\![C[p]]\!]$.

Proof. By structural induction on C. Each operator's denotation (C.1) is defined compositionally in its constituents' denotations, so substituting a denotationally equal constituent leaves the whole unchanged. The parallel case uses the disjoint-footprint hypothesis of the $\|$-typing rule (updates to disjoint cells commute). The iteration case uses the defining property of least fixed points: if the functionals induced by p and $T(p)$ are equal, then all finite approximants are equal (by the induction hypothesis), so their suprema—the least fixed points—are equal. ■

C.4 Theorem 3.4 (Optimization Soundness)

Proof. Read all equalities below as the partial-equivalence $\equiv$ of C.1 (equality on all states where both sides are defined), since the DCE rule preserves denotation only on the defined domain and can strictly enlarge it. The compiler applies a finite sequence of rewrites $O = R_k \circ ... \circ R_1$ (finiteness follows from compilation terminating; each pass is a bounded traversal). By induction on k: $k = 0$ is trivial. For the step, write $O = R \circ O'$. By the induction hypothesis $[\![O'(p)]\!] \equiv [\![p]\!]$. Two routes, by rule kind. **Fusion and peephole** are local subprogram rewrites: the atomic-rewrite lemma (C.2) preserves the denotation of the subprogram it rewrites, and the compositionality lemma (C.3) lifts the preservation through any surrounding composition context. **Dead-code elimination** is not local—liveness is computed over the whole surrounding program—so it is handled at pass level directly by C.2(ii)'s whole-program argument ($p \equiv p \setminus \{i\}$), not through the lifting template. Hence $[\![R(O'(p))]\!] \equiv [\![O'(p)]\!] \equiv [\![p]\!]$. ■

C.5 Theorem 3.5 (Pipeline Correctness)

Let s be a valid instruction sequence and S a schedule satisfying:

- **(S1)** every instruction is dispatched exactly once;
- **(S2)** for every dependency $(i_1, i_2) \in \mathit{Deps}$: $\mathit{start}(i_2) \geq \mathit{end}(i_1)$ (the constraint of Section 3.4.3);
- **(S3)** instructions sharing a parallel dispatch group are pairwise non-interfering (no data hazard, no resource conflict);
- **(S4)** commit is in program order.

Then executing s under S yields the same final state as sequential execution: $[\![s]\!]_S = [\![s]\!]_{seq}$.

Proof. Let $g_1, ..., g_m$ be the dispatch groups of S in commit order. By S4, groups commit in program order and group commits do not interleave, so the union $\bigcup_{j \leq t} g_j$ of committed groups is always a program-order prefix—"executing all instructions in those groups" below means executing that prefix sequentially. We show the invariant

$$I(t) : \text{after committing } g_1, ..., g_t,$$
$$\text{the architectural state equals the sequential state after}$$
$$\text{executing all instructions in those groups.}$$

- **Base** ($t = 0$): both sides are the initial state.
- **Step**: assume $I(t)$. Every instruction in g_{t+1} has its dependencies in earlier groups (by S2, since it is scheduled after them), which by $I(t)$ have completed and committed. Within the group, S3 gives pairwise non-interference: no instruction reads another group-member's output, and no two write the same location or resource. Hence the group's updates commute—for pure updates this is the diamond property, and for effectful instructions, S3's exclusion of shared resources makes their effects independent, so the combined effect is unique regardless of interleaving. The state after g_{t+1} is therefore independent of the group's internal interleaving and equals the sequential execution of the same instructions from the state at $I(t)$.
- By induction, $I(m)$ holds; by S1 no instruction was skipped or duplicated.

This completes the induction over all dispatch groups, hence the proof. ■

Corollary (dispatch folding). The ADVANCED and AGGRESSIVE levels fold instructions into shared dispatches (Section 5.3). A folded group satisfies S3 by construction: the grouping check admits only side-effect-free instructions whose dependencies and declared outputs are pairwise disjoint, so the group's members act on disjoint state components and their updates commute (Definition 3.6, Property 2). By Theorem 3.5 the fold changes the *dispatch count* but neither the semantics nor—consistent with the measurement of Section 5.3—the effective instruction count.

C.6 Theorem 3.6 (Pass-Pipeline Soundness)

Every optimization in the pass pipeline is a finite composition of rewrites covered by the atomic-rewrite lemma (C.2) or by the dispatch-folding corollary of Theorem 3.5 (C.5)—the grouping pass is a fold, not a local rewrite—and the argument of C.4 applies verbatim to each rewrite layer. ■

C.7 Scope Remark

Theorems 3.4–3.6 are *conditional*: they hold of transformations whose verification obligations (Sections 4.1.3 and 4.3.1) are discharged. As stated there, those obligations are **specified but not yet mechanically discharged**—the shipped optimizer registers no peephole rules (the peephole obligation is dormant, and the fusion/DCE/grouping passes are argued directly in C.2 and C.5 rather than through the symbolic-execution checker), and the symbolic-execution equivalence checker itself is a specification (Section 4.3.1). Where an obligation is discharged only on bounded state spaces (Section 3.3.3), the guarantee is correspondingly bounded: rewrites verified on the explored subset are sound there. No such rewrite is registered today; when one is, its application must be restricted to the explored subset (that restriction is a dormant obligation of the future rule set, not shipped behavior). What the shipped implementation does provide is empirical semantics-preservation at task level: the system-level demonstration of Section 5.8 re-executes an optimized stream and checks byte-identical output.

Observation model. The theorems' equality is *partial equivalence* (C.1: $[\![p]\!] = [\![q]\!]$ on states where both are defined), and the observation model should be read accordingly. **Observed:** the final state of declared state cells, the declared outputs (the instruction's product keys), and—for theorems covering effectful instructions (C.5)—the externally visible effects, in program order. **Deliberately not observed:** the *domain of definition* itself. Because equivalence is only partial, a rewrite may extend definedness—DCE is the canonical example (C.4): deleting an instruction that would have failed can turn a previously undefined run into a defined one. If program failure or exception behavior were part of the observation model, this would violate preservation; it is therefore stated explicitly that failure behavior is outside what Theorems 3.4–3.6 preserve, and any deployment

for which failures are observable (e.g., a pipeline contract that treats exceptions as data) needs a strengthened equivalence and must re-derive the theorems under it. Internal traces—the sequence of instructions executed—are likewise not observed; only the net transformation is.

Three status levels. Each mechanism in this paper falls into exactly one of three levels, which we keep separate throughout: *(i) specified*—the symbolic-execution equivalence checker, the pipeline scheduler's cycle-accurate extension, tree-pattern instruction selection (Section 4.1.2); *(ii) implemented with checks*—the optimizer passes (fusion, DCE, grouping) with their pass-level arguments in C.2/C.5, the runtime monitors, and differential unit tests pinning the fusion property this appendix argues (fused-composite execution equals constituent-sequence execution; the fusion gate refuses producer–consumer pairs; `tests/core/isa/test_fusion_differential.py`); *(iii) empirically validated*—trace-level replay identity on the mock (Section 5.4: optimized traces re-executed, final states identical in 5/5 seeds), task-level re-execution with byte-identical outputs (Section 5.8), and the caching/compilation measurements of Section 5 (including the exact-response-cache parity baseline of Section 5.2). A statement at one level is never used to imply a guarantee at a stronger level.

D Specified-Only Component Listings

This appendix carries the algorithm listings that the main paper specifies but does not ship (or ships dormant), moved from its Section 4 to respect the page limit. Each listing keeps its original framing; implementation status statements are unchanged. The shipped selection step (the heuristic name-overlap matcher) remains in the source module `agentflow.core.isa.instruction_selector`.

D.1 Specified Tree-Pattern Instruction Selection (MATCH-TREE)

Input: IR tree T, pattern set P; output: optimal pattern selection S.

```
Input: IR tree T, Pattern set P
Output: Optimal pattern selection S

function MATCH-TREE(T, P)
    costs[n] ← ∞ for all nodes n in T
    Let selected[n] = ∅ for all nodes n in T

    function MATCH-SUBTREE(node)
        if costs[node] ≠ ∞ then
            return costs[node]
        end if

        min_cost ← ∞
        best_pat ← null

        for each pattern p in P do
            if MATCHES(p, node) then
                curr_cost ← PATTERN-COST(p)
                for each child c of node not covered by p do
                    curr_cost ← curr_cost + MATCH-SUBTREE(c)
                end for
```

```
                if curr_cost < min_cost then
                    min_cost ← curr_cost
                    best_pat ← p
                end if
            end if
        end for

        costs[node] ← min_cost
        selected[node] ← best_pat
        return min_cost
    end function

    MATCH-SUBTREE(root(T))
    return (costs, selected)
end function
```

Theorem 4.1 (Selection Optimality, specified algorithm): given a set of context-local patterns P and an IR tree T, the MATCH-TREE algorithm computes the minimal covering cost; the covering itself is read off the `selected` table afterwards, by walking from the root and skipping subtrees spanned by the chosen pattern.

Proof: By induction on tree height, assuming a covering of T exists (if no pattern matches a node its cost is ∞ and no covering exists):

1. Base case (a leaf has no children): the recurrence degenerates to a plain argmin over the patterns matching the leaf, which is optimal for a single node.
2. Inductive step: For a node v with children $c_1, \ldots, c_k$:

$$\text{cost}(v) = \min_{p \in P} \left\{ p.\text{cost} + \sum_{\substack{c \in \text{children}(v) \\ c \notin \text{covered}(p)}} \text{cost}(c) \right\}$$

   By the induction hypothesis, each $\text{cost}(c)$ is the minimal covering cost of the corresponding subtree; the recurrence takes each candidate pattern's cost plus the (already optimal) costs of the children the candidate does not span, so the minimum over candidate patterns is the minimal covering cost at v—the minimum over disjoint pattern choices is optimal.

The recurrence sums only over children not covered by the candidate pattern p—summing over all children would double-count the cost of subtrees the pattern spans. Section 3.4.3's per-node argmin is therefore exact for single-node patterns; multi-node patterns require this covering recurrence.

The recurrence requires that a pattern's feasibility at a node not depend on the covers chosen for other subtrees; context constraints that couple sibling subtrees break this optimal-substructure property, which is why the statement restricts to context-local patterns. This theorem is a property of the specified algorithm, not of the shipped heuristic selector; no optimality claim is made for the implementation.

D.2 Specified Cycle-Accurate Pipeline Scheduler (SCHEDULE-PIPELINE)

Input: instruction sequence I; output: schedule S with cycle assignments.

```
Input: Instruction sequence I
Output: Schedule S with cycle assignments

function SCHEDULE-PIPELINE(I)
    D ← BUILD-DEPENDENCY-GRAPH(I)
    S ← empty schedule
    R ← priority queue of ready instructions

    while D not empty do
        c ← CURRENT-CYCLE()

        // Find newly ready instructions
        N ← GET-READY-NODES(D, c)
        ADD-TO-QUEUE(R, N)

        // Schedule instructions for current cycle
        while R not empty and HAS-RESOURCES(c) do
            i ← POP-HIGHEST-PRIORITY(R)
            slot ← ALLOCATE-RESOURCES(i, c)
            ADD-TO-SCHEDULE(S, i, slot)
            UPDATE-DEPENDENCIES(D, i)
        end while
    end while

    VERIFY-SCHEDULE(S)
    return S
end function
```

D.3 Specified Dynamic Pipeline Controller (PipelineController)

```
class PipelineController:
    def __init__(self):
        self.hazard_detector = HazardDetector()
        self.resource_monitor = ResourceMonitor()
        self.performance_tracker = PerformanceTracker()

    async def execute_pipeline(self,
                               instruction_stream: AsyncIterator[Instruction]
                              ) -> AsyncIterator[Result]:
        async for inst in instruction_stream:
            # Hazard detection
            while self.hazard_detector.has_hazard(inst):
                await self.hazard_detector.wait_resolution()

            # Resource allocation
```

```
            resources = await self.resource_monitor.allocate(inst)

            try:
                # Execution
                result = await self.execute_instruction(inst, resources)
                # Performance tracking
                self.performance_tracker.record(inst, result)
                yield result
            finally:
                self.resource_monitor.release(resources)
```

D.4 Specified Hot-Path Detector (DETECT-HOT-PATH)

Input: execution path P, threshold θ, EMA decay γ ($\gamma = 2/(w+1)$ for window w; γ is unrelated to the α stability bound of the main paper's Section 3.4.4); output: Boolean indicating if P is hot.

```
Input: Execution path P, Threshold θ, EMA decay γ (γ = 2/(w+1) for window w; γ is unrelated to t
Output: Boolean indicating if P is hot

function DETECT-HOT-PATH(P, θ, γ)
    Let counter be an exponential moving average with decay γ

    function UPDATE-COUNTERS(path)
        h ← COMPUTE-PATH-HASH(path)
        counter[h] ← counter[h] × (1 - γ) + 1

        if IS-PATH-HOT(path) then
            TRIGGER-OPTIMIZATION(path)
        end if
    end function

    function IS-PATH-HOT(path)
        h ← COMPUTE-PATH-HASH(path)
        freq ← counter[h] / Σ_{h'} counter[h']   // recent execution share, not cumulative total
        stab ← COMPUTE-STABILITY(path)
        return freq > θ and stab > STABILITY-THRESHOLD
    end function

    UPDATE-COUNTERS(P)
    return IS-PATH-HOT(P)
end function
```

D.5 Specified Trace Formation (BUILD-TRACE)

Input: hot path H; output: optimized trace T.

```
Input: Hot path H
Output: Optimized trace T
```

```
function BUILD-TRACE(H)
    T ← empty trace

    function SHOULD-INCLUDE(block)
        return EXECUTION-FREQUENCY(block) > THRESHOLD and
               PREDICTABLE-BRANCHES(block)
    end function

    function SHOULD-SPLIT(block)
        return HAS-SIDE-EXIT(block) or
               EXECUTION-FREQUENCY(block) < MIN-FREQ
    end function

    for each block B in H do
        if SHOULD-INCLUDE(B) then
            APPEND-TO-TRACE(T, B)
        else
            break   // any block not selected terminates the trace
        end if
    end for

    VERIFY-TRACE-PROPERTIES(T)
    return T
end function
```

D.6 Specified Symbolic-Equivalence Checker (VERIFY-TRANSFORMATION)

Input: original CFG O, transformed CFG T; output: Boolean indicating semantic equivalence.

```
Input: Original CFG O, Transformed CFG T
Output: Boolean indicating semantic equivalence

function VERIFY-TRANSFORMATION(O, T)
    // Build symbolic execution paths
    P₁ ← SYMBOLIC-EXECUTE(O)
    P₂ ← SYMBOLIC-EXECUTE(T)

    function VERIFY-EQUIVALENCE(path₁, path₂)
        // σ ranges over the symbolic states Σ explored on both paths
        for each symbolic state σ ∈ Σ do
            if EVALUATE(path₁, σ) ≠ EVALUATE(path₂, σ) then
                return false
            end if
        end for
        return true
    end function

    // Paths must pair one-to-one; a length mismatch means the
```

```
    // transformation added or removed a symbolic path — reject.
    if LENGTH(P₁) ≠ LENGTH(P₂) then
        return false
    end if

    // Verify all path pairs, paired by the transformation's path map
    for each (p₁, p₂) in PATH-MAP(P₁, P₂) do
        if not VERIFY-EQUIVALENCE(p₁, p₂) then
            return false
        end if
    end for

    return true
end function
```

D.7 Specified Performance Monitor

```
class PerformanceMonitor:
    def monitor_execution(self,
                          instruction: Instruction,
                          context: Context) -> Metrics:
        start = time.perf_counter_ns()
        result = self.execute(instruction, context)
        end = time.perf_counter_ns()

        return Metrics(
            latency=(end - start),
            resource_usage=self.measure_resources()
        )
```

D.8 Shipped Peephole Optimizer (Empty Pattern Table)

```
class PeepholeOptimizer:
    def optimize(self, instructions: List[Instruction]) -> List[Instruction]:
        patterns = self.load_patterns()
        optimized = []
        i = 0
        while i < len(instructions):
            matched = False
            for pattern in patterns:
                if pattern.matches(instructions[i:i+pattern.length]):
                    optimized.extend(pattern.replace(instructions[i:i+pattern.length]))
                    i += pattern.length
                    matched = True
                    break
            if not matched:
                optimized.append(instructions[i])
                i += 1
```

```
        return optimized
```

D.9 Specified Context-Aware Optimization Sketch (OPTIMIZE-CONTEXT)

Programmatic sketch from the main paper's Section 6.3.2, item 2 (future work; the performance model and the variant generator are future work, not implemented machinery):

```
function OPTIMIZE-CONTEXT(I, C)
    Let M be a context-performance model, R the rewrite rules
    for each context c in C do
        if PREDICT-PERFORMANCE(M, I, c) < THRESHOLD then
            candidates ← GENERATE-VARIANTS(I, R)
            I ← SELECT-BEST(candidates, c)
            UPDATE-MODEL(M, I, c)
        end if
    end for
    return I
end function
```

D.10 Shipped Coordination Flow and Fusion Pass Listings

The coordination flow implemented by `AgentCoordinator` (authenticate, authorize, select by capability, plan stages with fallback, execute, record history), referenced by the main paper's Section 5.6:

```
Input: Agent set A, Task list τs
Output: Coordinated solutions S

function COORDINATE-AGENTS(A, τs)
    for each task T in τs do
        AUTHENTICATE(task issuer)              // SecurityMonitor
        AUTHORIZE(task, role policy)           // SecurityMonitor
        ag ← SELECT-BY-CAPABILITY(A, T)        // CapabilityProfile (Section 3.5)
        plan ← BUILD-EXECUTION-PLAN(T)         // stages + coordination rules
        task-outcome ← success
        for each stage in plan do
            stage-outcome ← EXECUTE-STAGE(stage, ag)
            if stage-outcome failed and fallback(stage) exists then
                stage-outcome ← EXECUTE-FALLBACK(stage)
            end if
            if stage-outcome failed then
                task-outcome ← failed
                break
            end if
        end for
        RECORD-HISTORY(T, task-outcome)
    end for
end function
```

The shipped instruction-combining (fusion) pass of the main paper's Section 4.1.3, also in the source module `agentflow.core.isa.instruction_optimization`:

```
def combine_instructions(self, insts: List[Instruction]) -> List[Instruction]:
    combined = []
    current_group = []

    for inst in insts:
        if self.can_combine(current_group, inst):
            current_group.append(inst)
        else:
            if current_group:
                combined.append(self.merge_group(current_group))
            current_group = [inst]

    if current_group:
        combined.append(self.merge_group(current_group))
    return self.verify_combinations(combined)
```

E Formal Sketches for Future Directions

Sketches referenced by the main paper's Section 6.3; all are future work with no current implementation.

E.1 Stated Sketches

Compositional reasoning (Section 6.3.1, item 2): a parallel-composition rule

$$\frac{\{P_1\}S_1\{Q_1\} \quad \{P_2\}S_2\{Q_2\} \quad \text{disjoint}(S_1, S_2)}{\{P_1 \wedge P_2\}S_1 \parallel S_2\{Q_1 \wedge Q_2\}}$$

whose disjointness side condition mirrors the $\parallel$-typing rule of the main paper's Definition 3.6.

Capability type theory (Section 6.3.1, item 3):

```
Type Capability = {
  Basic: {Perception, Action, Memory}
  Composite: Basic → Basic
  Emergent: P(Composite)
}
```

with typing rules

$$\frac{\Gamma \vdash a : \text{cap}_1 \quad \Gamma \vdash b : \text{cap}_2}{\Gamma \vdash a \circ b : \text{cap}_1 \otimes \text{cap}_2}$$

Resource consumption model (Section 6.3.3, item 2): $R(i) = \alpha C_{cpu}(i) + \beta C_{mem}(i) + \gamma C_{io}(i)$ with platform-specific coefficients.

Information flow control (Section 6.3.6, item 1):

$$\text{Secure}(P) \iff \forall s_1, s_2 : L(s_1) = L(s_2) \implies L([\![P]\!]s_1) = L([\![P]\!]s_2)$$

where L assigns security levels to states and outputs (classical noninterference, terminating runs only).

F Artifact Regeneration Commands

Exact command lines mirrored from the repository's `REPRODUCING.md`, referenced by the main paper's Artifact Availability section.

F.1 Section 5.2–5.5 microbenchmarks

```
./venv/bin/python examples/benchmarks/isa_microbench.py \
    --experiments cache compiler dynamic ablation --seeds 5
```

F.2 Section 5.7 real-gateway cache run

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64 \
    --output examples/benchmarks/isa_real_llm_results.json
```

(requires a reachable gateway; the base URL, API key, and CA certificate are passed per deployment). Any Anthropic-protocol gateway works—configure `ANTHROPIC_BASE_URL` and `ANTHROPIC_AUTH_TOKEN`; the paper's run used a local alias for a hosted GLM reasoning model:

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64
```

F.3 Section 5.7.1 real-gateway compiler/hot-path harness

```
./venv/bin/python examples/paper_real_llm_compiler_test.py \
    --backend real --scale paper
```

(the `mock` backend reproduces the offline variant; `--scale paper` selects the 50-realization, 40-instance protocol of the paper run).

F.4 Section 5.8 four-task demonstration

```
./venv/bin/python examples/system_task_demo.py \
    --seed 42 --files 6 --lines-per-file 400 --repeats 12 \
    --hot-threshold 5 --output examples/system_task_results.json
```

with test suite `pytest tests/system/test_system_task_demo.py`.

F.5 Artifact Contents Inventory

Per-file inventory of the artifact (mirrored from `REPRODUCING.md`):

- the AgentISA implementation (`agentflow.core.isa`);
- the microbenchmark suite (`examples/benchmarks/isa_microbench.py`);

- the real-LLM benchmark (`examples/benchmarks/isa_real_llm_bench.py`);
- the system-level task demonstration (`examples/system_task_demo.py`, raw results in `examples/system_task_results.json`);
- the four-task workload characterization harness (`examples/system_task_workload.py`, tests in `tests/system/test_system_task_workload.py`);
- the real-gateway compiler/hot-path harness (`examples/paper_real_llm_compiler_test.py`), its offline-mock and real-gateway results (`examples/paper_real_llm_compiler_results_mock.json`, `examples/paper_real_llm_compiler_results_real.json`), and tests (`tests/system/test_real_llm_compiler_test.py`);
- the raw results (`examples/benchmarks/isa_microbench_results.json` and `examples/benchmarks/isa_real_llm_results.json`);
- the 43 multi-agent functional tests (`tests/core/isa/test_coordination.py`, `tests/core/instructions/te agent_coordination.py`).

G Priced-Comparison Protocol and Table

Full protocol of the priced comparison summarized by the main paper's Section 5.7.1; all costs are MODELED from a pricing overlay on measured tokens.

A priced comparison (costs MODELED—computed from a pricing overlay rather than observed—on measured tokens). The harness prices the run's measured token counts under three modes: *no optimization* (every realization re-queries at full price), *AgentISA result caching* (first occurrences only—the measured behavior of both arms), and a *provider prompt cache* (MODELED: every realization queries, but repeat prompts bill input tokens as prompt-cache reads at 0.1x the input price—configurable—mirroring Anthropic's published prompt-caching read discount; completions always bill at full price). Two simplifications—omitting Anthropic's 1.25x cache-write premium and 1024-token minimum cacheable prefix—both favor the provider-cache baseline, so the comparison is conservative for us. List prices are parameterized stand-ins ($3/M input, $15/M output by default) because the gateway's actual pricing is unknown. Token counts are measurements, the pricing overlay is a model, and no dollar-savings claim is made:

Table 2: Measured calls and tokens with a MODELED pricing overlay (real-gateway run, s = 1.0, 50 realizations; the price column is a model, not a quote).

Pricing mode	Calls	Input tokens (full / cache-read)	Output tokens	Modeled cost
No optimization	50	9,018 / 0	25,600	$0.4111
AgentISA result caching	22	4,027 / 0	11,264	$0.1810
Provider prompt cache (MODELED)	50	4,027 / 4,991	25,600	$0.3976

On this run's token mix the result cache cuts modeled cost by 56.0%—it removes whole calls, completions included—while the modeled provider prompt cache cuts only 3.3%, because the reasoning model's completion tokens dominate and a discount on input reads cannot touch them; the ranking of the two caching strategies is a function of the input/output token mix, not a constant. On the deterministic mock at paper scale (200 tasks × 1 seed × 3 Zipf levels, template-accounted tokens, same

modes and prices) the result cache cuts modeled cost by 80.1%, 81.6%, and 85.1% at s = 0.5, 1.0, 1.5, while the modeled provider prompt cache cuts 28.9–31.4%—there the short completions do not dominate, so the input-side discount matters more; both columns remain MODELED-on-measured in both cases. GPTCache, an external cache layer, is explicitly out of scope; no head-to-head against it is claimed.

H Four-Task Characterization — Full Detail

Task inventory, per-task instruction streams, configuration, single-task walk-through, and both result tables of the main paper's Section 5.8; numbers identical to the main paper.

The four tasks:

- the original **service-log triage** (linear, 7 instructions), detailed below;
- **csv-etl-anomaly** (fan-out, 8): CSV shard ingest, schema guard, parse, per-column statistics, row signatures and anomaly triage, aggregate, write;
- **report-merge** (wide fan-in, 7): five-report ingest, header guard, dedupe, per-line signatures, cacheable consistency check, merge, write;
- **regression-diff** (two-branch convergence, 8): parallel baseline/candidate ingests, guard, normalize, diff, classify hunks, aggregate, write.

The triage task's instruction stream (seven instructions over sandbox-resident files): `ingest_logs` reads six 400-line sandbox log files (side-effectful); `check_syntax` is a defensive format guard that never rejects here (a *planted dead guard*, standing in for real pipelines' belt-and-braces checks); `normalize_entries` parses and normalizes lines; `extract_incidents` lifts them into incident records; `classify_signature` triage-classifies each incident (LLM-type, cacheable, backed by a deterministic call-counting rule stand-in); `aggregate_triage` summarizes by severity; `write_report` writes the report (side-effectful).

Configuration. Seed 42; 6 files × 400 lines; 12 repeated realizations of the task; hot-path threshold 5. Every phase is deterministic and re-runnable from the repository (Artifact Availability).

What the triage walk-through shows.

- **Static compilation with structural validation.** NONE compiles to 7 effective instructions / 7 dispatches; AGGRESSIVE to 6 / 6 (14.3%): the dead guard fuses with the following pure step rather than being dropped. The walk-through *asserts the compiled stream's dataflow validity*—every step whose output is consumed downstream must survive compilation, the only eliminable step being the guard—so the shrinkage cannot come from dropping live producers.
- **Reuse across repeated realizations.** The first realization makes 50 (simulated) model calls; all eleven subsequent realizations make zero (result cache hit rate 91.7% on the classification step); all 12 reports are byte-identical.
- **Closed-loop hot-path adoption.** The stream crosses the hot-path threshold at the sixth recording (the counter must strictly exceed it and recording follows execution, so adoption lands at realization threshold+2); the runtime *adopts* the optimized form automatically, repeats 7–12 execute the shrunken 6-instruction stream (instruction executions per realization: `[7,7,7,7,7,7,6,6,6,6,6,6]`), and on a fresh workspace the optimized form reproduces the report byte-for-byte—semantics-preserving on the real task, not only on synthetic sequences.
- **Full-stack determinism.** A completely fresh stack—new instructions, new cache, new workspace, same seed—reproduces the report byte-for-byte (SHA-256 equality).

Configuration. Seed 42; 6 files × 400 lines; 12 repeated realizations of the task; hot-path threshold 5. Every phase is deterministic and re-runnable from the repository (Artifact Availability).

What the triage walk-through shows.

- **Static compilation with structural validation.** NONE compiles to 7 effective instructions / 7 dispatches; AGGRESSIVE to 6 / 6 (14.3%): the dead guard fuses with the following pure step rather than being dropped. The walk-through *asserts the compiled stream's dataflow validity*—every step whose output is consumed downstream must survive compilation, the only eliminable step being the guard—so the shrinkage cannot come from dropping live producers.
- **Reuse across repeated realizations.** The first realization makes 50 (simulated) model calls; all eleven subsequent realizations make zero (result cache hit rate 91.7% on the classification step); all 12 reports are byte-identical.
- **Closed-loop hot-path adoption.** The stream crosses the hot-path threshold at the sixth recording (the counter must strictly exceed it and recording follows execution, so adoption lands at realization threshold+2); the runtime *adopts* the optimized form automatically, repeats 7–12 execute the shrunken 6-instruction stream (instruction executions per realization: `[7,7,7,7,7,7,6,6,6,6,6,6]`), and on a fresh workspace the optimized form reproduces the report byte-for-byte—semantics-preserving on the real task, not only on synthetic sequences.
- **Full-stack determinism.** A completely fresh stack—new instructions, new cache, new workspace, same seed—reproduces the report byte-for-byte (SHA-256 equality).

The four-task characterization. All four tasks run the identical closed-loop battery (`examples/system_task_workload.py`, `characterize()`, 12 realizations each, hot-path threshold 5, one planted dead guard and one cacheable LLM-type step each, the latter backed by a deterministic rule stand-in counting simulated model calls):

Structure (compile-time):

Table 3: Compile-time structure of the four tasks (12 realizations each; shrink is the NONE → AGGRESSIVE effective-instruction reduction).

Task (shape)	Instrs	Effective count NONE → AGGRESSIVE	Shrink	Dead guard
log triage (linear, 7)	7	7 → 6	-14.3%	`check_syntax`
csv-etl-anomaly (fan-out, 8)	8	8 → 6	-25.0%	`check_schema`
report-merge (wide fan-in, 7)	7	7 → 6	-14.3%	`check_headers`
regression-diff (two-branch, 8)	8	8 → 7	-12.5%	`check_formats`

Execution (runtime, 12 realizations each):

Table 4: Runtime behavior of the four tasks (12 realizations each; classification-type steps are backed by deterministic rule stand-ins).

Task	Calls 1st / re-runs	Cache hit	Hot-path adopted	Reports byte-identical
log triage	50 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
csv-etl-anomaly	14 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
report-merge	16 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
regression-diff	240 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)